

Universidad del valle de Guatemala
Programación orientada a objetos
Proyecto. Entrega 2
David Adriel Aguirre Leonzo – 261134
Allan Enrique Ronneseth Quemé - 26493
Fabio Roberto Calvo Lemus – 262256



1. Requisitos funcionales:

Los siguientes son los requisitos funcionales identificados para el desarrollo del mod de Minecraft que añade contenido radioactivo a la dimensión del End:

RF-01: Generar Bloque Radioactivo en el End

El sistema debe permitir la generación automática del bloque radioactivo en islas específicas del End con una distribución equilibrada.

RF-02: Aplicar Efectos de Radiación al Jugador

Cuando el jugador se acerque al bloque radioactivo, debe recibir efectos visuales y de juego como daño gradual, efectos de radiación, y partículas.

RF-03: Permitir Minería del Bloque Radioactivo

El jugador debe poder minar el bloque radioactivo con herramientas adecuadas. El bloque debe droppear un objeto recolectable.

RF-04: Sistema de Almacenamiento del Material Radioactivo

El material radioactivo debe poder ser recolectado y almacenado en el inventario del jugador con propiedades visuales distintivas.

RF-05: Implementar Sistema de Daño por Radiación

El bloque radioactivo debe causar daño gradual al jugador en su proximidad.

RF-06: Añadir Efectos Visuales y de Partículas

El bloque debe emitir partículas radioactivas visibles y tener un brillo característico.

RF-07: Implementar Sonidos Específicos

El bloque radioactivo debe tener sonidos característicos al acercarse y ser minado.

RF-08: Crear Modelo y Textura del Bloque

El bloque debe tener un modelo 3D único y una textura personalizada coherente con la estética del End.

RF-09: Sistema de Configuración

El mod debe permitir la configuración de parámetros como frecuencia de aparición y radio de efecto.

RF-10: Compatibilidad con NeoForge

Todo el código debe ser compatible con NeoForge y Minecraft 1.21+.

2. Priorización de requisitos funcionales:

Los requisitos han sido priorizados según su importancia. Los de prioridad 1 son críticos:

Prioridad 1 (Críticos): RF-01, RF-03, RF-04, RF-08, RF-10

Estos son los requisitos esenciales sin los cuales el mod no puede funcionar.

Prioridad 2 (Altos): RF-02, RF-05, RF-06

Requisitos importantes para la experiencia de juego pero pueden refinarse después.

Prioridad 3 (Medios): RF-07, RF-09

Mejoras visuales y configuración avanzada que pueden implementarse posteriormente.

3. IDENTIFICACIÓN Y DESCRIPCIÓN DE CLASES

El sistema utiliza el patrón Modelo-Vista-Controlador (MVC). Se utilizan las siguientes clases:

MODELO

1. RadiationEffect

Propósito: Representar el efecto de radiación que afecta a las entidades.

Atributos: Categoría del efecto e intensidad/duración.

Responsabilidades: Determinar cuándo se aplica la radiación y ejecutar sus efectos sobre las entidades.

2. RadioactiveBlock

Propósito: Representar un bloque radiactivo dentro del mundo.

Atributos: properties.

Responsabilidades: Crear la entidad del bloque y controlar la actualización periódica mediante getTicker().

3. RadioactiveBlockEntity

Propósito: Representar y controlar el comportamiento de un bloque radiactivo.

Atributos: BlockPos y BlockState.

Responsabilidades: Ejecutar el método tick() para realizar periódicamente la lógica de radiación.

4. RadioactiveItem

Propósito: Representar un objeto radiactivo que puede estar en el inventario del jugador.

Atributos: properties.

Responsabilidades: Ejecutar inventoryTick() y aplicar los efectos correspondientes cuando el objeto se encuentra en el inventario.

VISTA

1. ModCreativeTabs

Propósito: Mostrar y organizar los elementos del mod en el inventario creativo.

Atributos: CREATIVE_MODE_TABS y EXOTIC_ORES_TAB.

Responsabilidades: Crear la pestaña del mod y mostrar los bloques y objetos disponibles.

2. ClientModEvents

Propósito: Gestionar los elementos relacionados con la presentación del mod en el cliente.

Responsabilidades: Ejecutar onClientSetup() y realizar las configuraciones visuales necesarias del lado del cliente.

CONTROLADOR

1. ExoticOres

Propósito: Actuar como clase principal del sistema.

Atributos: `LOGGER` y `MODID`.

Responsabilidades: Inicializar el mod y coordinar el registro de bloques, objetos, efectos y demás componentes.

2. ModBlocks

Propósito: Gestionar los bloques del sistema.

Atributos: `BLOCKS` y `THORITE_ORE`.

Responsabilidades: Registrar los bloques y conectarlos con el resto del sistema.

3. ModItems

Propósito: Gestionar los objetos del sistema.

Atributos: `ITEMS`, `THORITE_ORE_ITEM` y `THORITE_SHARD`.

Responsabilidades: Registrar los objetos y permitir su utilización dentro del juego.

4. ModEffects

Propósito: Gestionar los efectos personalizados.

Atributos: `MOB_EFFECTS` y `RADIATION`.

Responsabilidades: Registrar y proporcionar el efecto de radiación.

5. ModBlockEntities

Propósito: Gestionar las entidades asociadas a los bloques.

Atributos: `BLOCK_ENTITIES` y `RADIOACTIVE_BLOCK_ENTITY`.

Responsabilidades: Registrar la entidad del bloque radiactivo y relacionarla con `RadioactiveBlock`.

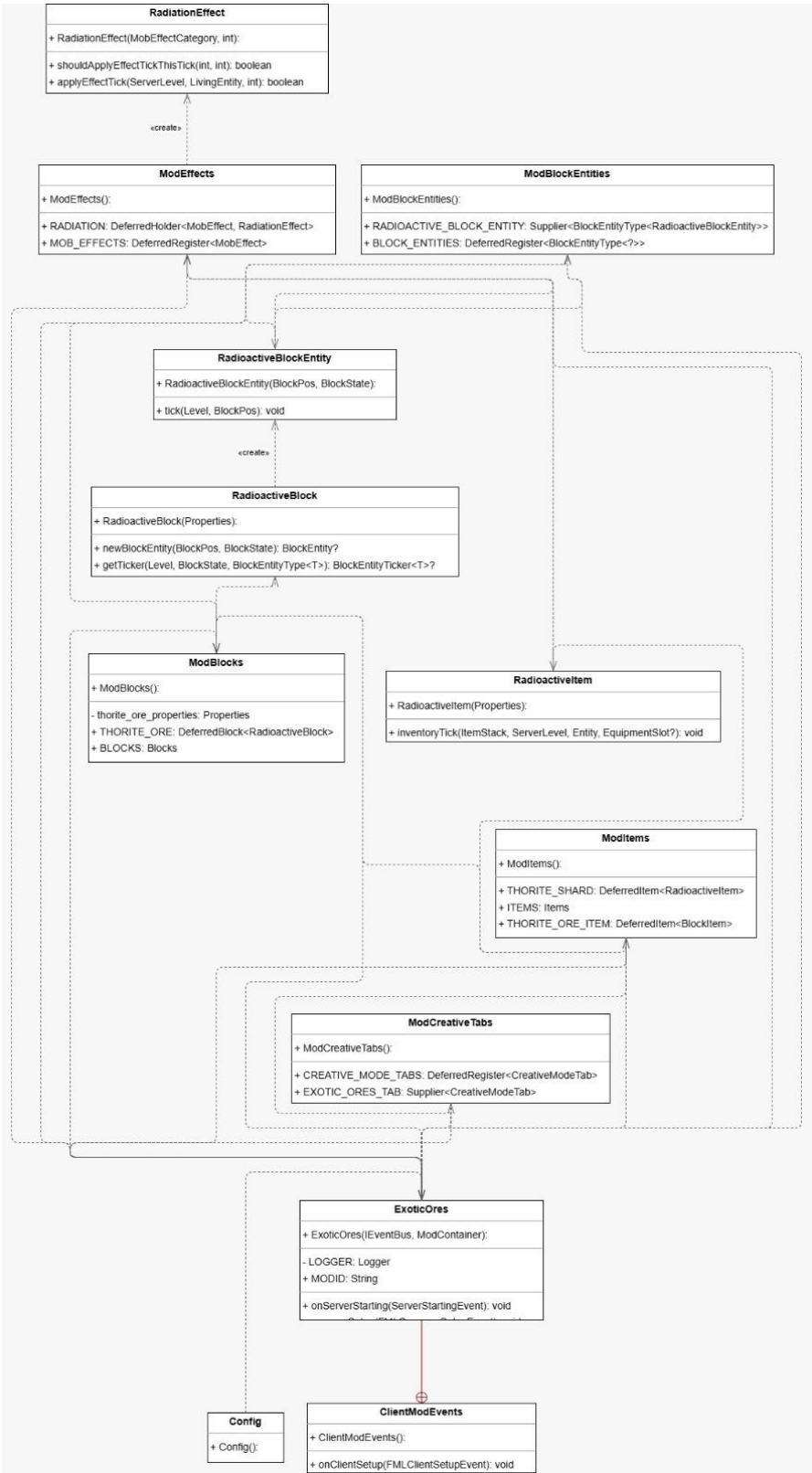
CLASE AUXILIAR

1. Config

Propósito: Almacenar la configuración general del sistema.

Responsabilidades: Definir y proporcionar parámetros de configuración utilizados por las demás clases.

UML



4. Tecnologías empleadas:

NeoForge:

NeoForge es un framework y mod loader de código abierto para Minecraft: Java Edition que fue creado como una continuación moderna de Minecraft Forge. Proporciona herramientas, APIs y sistemas de construcción que permiten a los desarrolladores modificar el comportamiento del juego, añadir nuevas mecánicas de jugabilidad e integrar contenido personalizado en mundos de Minecraft. La arquitectura de NeoForge opera en cinco capas que facilitan la extensión profunda del juego.

Para este proyecto, NeoForge es ideal porque proporciona APIs ricas que permiten modificaciones profundas, mantiene compatibilidad con Minecraft 1.21+, ofrece un ecosistema activo de mods, y soporta sistemas de eventos que facilitan la implementación de efectos de radiación. (*The NeoForged Project: Installer and Modder Information*, 2026)

BlockBench:

BlockBench es una aplicación de modelado 3D gratuita y de código abierto diseñada específicamente para crear modelos de baja poligonización, particularmente adaptada para Minecraft. Permite a los usuarios diseñar modelos desde cero o importarlos en juegos con facilidad, proporcionando herramientas optimizadas para el estilo único de bloques de Minecraft. BlockBench se utiliza en este proyecto para diseñar el modelo 3D del bloque radioactivo, crear texturas personalizadas, exportar el modelo en formato compatible con NeoForge, y potencialmente añadir animaciones. (*Blockbench: Modeling, Texturing & Animating*, n.d.)

Gradle:

Gradle es una herramienta de automatización de construcción de código abierto y de gestión de dependencias que apoya el desarrollo de software en múltiples lenguajes, incluyendo Java. Permite a los desarrolladores automatizar tareas en el proceso de desarrollo como compilación, empaquetamiento, pruebas, despliegue y publicación.

En este proyecto, Gradle es esencial para gestionar las dependencias de NeoForge, compilar el código fuente, empaquetar el mod en un JAR, y automatizar pruebas. (Karkori, n.d.)

IntelliJ:

IntelliJ IDEA es un Entorno Integrado de Desarrollo profesional para Java desarrollado por JetBrains. Destaca como herramienta principal para programadores con capacidades avanzadas de navegación de código, refactorización integrada, asistencia de codificación inteligente y herramientas integradas para pruebas y perfilado.

IntelliJ IDEA es utilizado en este proyecto para desarrollar el código Java del mod, integración nativa con Gradle, depuración y pruebas, y refactorización de código. (Wikipedia contributors, 2026)

Referencias:

Blockbench: modeling, texturing & animating. (n.d.). Bedrock Wiki.
<https://wiki.bedrock.dev/guide/blockbench>

Karkori, M. (n.d.). Gradle para Principiantes 2025: Tu Guía Definitiva para Java y Spring Boot. *Mostapha.dev Blog*. <https://www.mostapha.dev/blog/gradle-principiantes-2025-guia-instalacion-uso-java-spring-boot>

The NeoForged Project: Installer and modder information. (2026, May 11).
<https://neoforged.net/>

The End (biome) – Minecraft Wiki. (n.d.). Minecraft Wiki.
[https://minecraft.wiki/w/The_End_\(biome\)](https://minecraft.wiki/w/The_End_(biome))

Wikipedia contributors. (2026, August 8). *IntelliJ IDEA*. Wikipedia.
https://en.wikipedia.org/wiki/IntelliJ_IDEA

5. Planificación y gestión de tareas.

Tarea 1: Clase RadioactiveBlock (Fabio Calvo) - Prioridad Crítica

Descripción: Implementación de la clase base del bloque radioactivo.

Horas estimadas: 4.

Fecha límite: 12 de agosto de 2026.

Tarea 2: Diseño de Textura en BlockBench (David Aguirre) - Prioridad Crítica

Descripción: Creación del modelo 3D y texturas del bloque radioactivo.

Horas estimadas: 4.

Fecha límite: 12 de agosto de 2026.

Tarea 3: Integración de Texturas (David + Fabio) - Prioridad Crítica

Descripción: Integración del modelo y texturas en el código del bloque.

Horas estimadas: 4.

Fecha límite: 12 de agosto de 2026.

Tarea 4: Sistema de Generación en el End (Fabio Calvo + David Aguirre + Allan) - Prioridad Crítica

Descripción: Implementación de la generación automática del bloque en islas del End.

Horas estimadas: 5.

Fecha límite: 12 de agosto de 2026.

Tarea 5: Sistema de Daño por Radiación (Fabio Calvo) - Prioridad Alta

Descripción: Implementación de daño y efectos de radiación a jugadores cercanos.

Horas estimadas: 12.

Fecha límite: 12 de agosto de 2026.

Tarea 6: Pruebas y Debugging (Fabio + Allan) - Prioridad Alta

Descripción: Pruebas funcionales y corrección de errores del mod.

Horas estimadas: 3.

Fecha límite: 12 de agosto de 2026.

Tarea 7: Documentación UML (Fabio + Allan) - Prioridad Alta

Descripción: Creación del diagrama UML de clases del proyecto.

Horas estimadas: 4-5.

Fecha límite: 12 de agosto de 2026.

Tarea 8: Documentación Final (Allan Ronneseth) - Prioridad Crítica

Descripción: Compilación y revisión de toda la documentación de la segunda entrega.

Horas estimadas: 3.

Fecha límite: 12 de agosto de 2026.

6. Registro de tiempo

Fecha	Inicio	Fin	Tiempo Interrupción (min)	Delta Tiempo (min)	Tarea	Comentarios
12/8/2026	3pm	5pm	20	100	Clase RadioactiveBlock y funcionalidad	Fue difícil de comprender.
12/8/2026	3pm	7pm	0	540	Diseño de Textura en BlockBench	Blockbench fue una experiencia interesante
12/8/2026	4pm	9pm	60 min	360	Integración de Texturas	Fue estresante
12/08/2026	4pm	8pm	30	210	Sistema de Generación en el End	Tomamos un descanso
12/8/2026	6pm	6:30pm	0	30	Sistema de Daño por Radiación	Sencillo de implementar.
12/08/2026	2pm	4pm	0	120	Pruebas y Debugging	El API dio algunos problemas.
12/08/2026	7pm	9pm	0	120	Documentación UML	Tarea colaborativa interesante, ya que Fabio nos enseña en el proceso.
12/08/2026	5pm	11pm	100	260	Documentación Final	Hay que hacer la cena (foraneo)

